

Source-Grounded Constraint Validation for Document-Level Knowledge Graph Repair

Hua Peng^{1†}, Zifu Tao^{2†}, Yu Guo¹, Qiang Zhou², Tian Zhou^{3*},
Bo Han³

¹China Mobile System Integration Co. Ltd., China.

²Tongji University, China.

³Xi'an Jiaotong University, China.

*Corresponding author(s). E-mail(s): tianzhou@xjtu.edu.cn;

Contributing authors: penghua@cmict.chinamobile.com;

2253323@tongji.edu.cn; guoyu1@cmict.chinamobile.com;

zhouqiang@cmict.chinamobile.com; bohan@xjtu.edu.cn;

[†]These authors contributed equally to this work.

Abstract

Knowledge graphs extracted from documents can contain missing fields and incorrect values. We study repair using source text, a field schema, and a language model. The pipeline normalizes the input, generates a candidate graph, and checks its heads, relations, source spans, and field counts. On 225 document graphs with 450 controlled defects, Diagnosis + Gate repairs 98.00% of defects. On 225 actual extraction outputs, it raises triple F1 from 87.99% to 92.48%. Direct field copying recovers all references in this serialized-record setting. We also study independently annotated receipt transcripts. An initial 60-document test identifies field-selection and text normalization errors. A follow-up uses 20 development documents and a frozen 60-document test to compare shared field definitions, a field-evidence index, and SHACL context. On this new test, extracted graphs reach 78.33% F1; Simple, Evidence Index, and SHACL with filtering reach 78.75%, 83.51%, and 78.33%. The paired Evidence Index–Simple difference is 4.76 points (95% CI: 2.50 to 7.26). The matched comparisons measure the added effects of evidence context and candidate filtering within schema-aware repair. Code, prompts, predictions, and annotation materials are available for further evaluation.

Keywords: Knowledge Graph Repair, Source Grounding, Constraint Validation, Document Extraction

1 Introduction

Knowledge graphs extracted from documents often contain missing fields, incorrect values, duplicate triples, and schema violations. These errors reduce the quality of information retrieval and decision support (Ji et al. 2022; Paulheim 2017). In many extraction workflows, the source text remains available and provides evidence for checking and repairing the graph.

Schema checks and language models serve different roles in this process. A schema check can identify an invalid relation or duplicate value. A language model can read the source and recover a missing field, but it can also change a correct value or add an unsupported fact. Work on KG quality and SHACL validation provides tools for checking graph constraints (Zaveri et al. 2016; Kontokostas et al. 2014; Knublauch and Kontokostas 2017). Lin et al. combine constraints and graph context in LLM repair prompts (Lin et al. 2025). These approaches motivate a practical question: how much do diagnostic context and output validation each contribute to source-based repair?

We study document-field graphs with a declared document node, a set of allowed relations, and at most one value per relation. Our pipeline first normalizes the input graph. A language model then proposes a complete graph using the source text and an optional diagnostic report. A deterministic filter checks each proposed triple for a valid head, an allowed relation, a value present in the source, and compliance with the one-value-per-field rule. It records a reason for each rejected triple.

The evaluation separates generation from validation. We compare base, diagnostic, and SHACL contexts under the same instructions and completion budget. Each response is scored before and after filtering. This design measures the effect of context on generation and the effect of filtering on a fixed set of candidates. We also compare a sequential selector and measure the cost of updating diagnostic reports.

We use controlled defects and actual extraction outputs from three Chinese record collections. Their source text lists the target fields explicitly, and a field-copy parser recovers all reference graphs. We therefore add an external test on SROIE receipt transcripts, whose text and human field labels are stored separately. The two settings let us compare behavior on serialized records and independently annotated document text. An error analysis then guides a small follow-up: shared field definitions and a deterministic index of source evidence lines. We evaluate this variant on a separate development set and freeze it before a new document test.

The paper provides three main results:

- A source-based candidate-validation procedure with explicit input requirements, field constraints, and rejection logs.
- A repair evaluation covering controlled defects, model extraction errors, and external receipt text, with measures of fact recovery, preservation, and harmful edits.
- A matched comparison of diagnostic context, field-evidence lines, SHACL context, and candidate filtering, supported by per-document predictions, statistical comparisons, and execution traces.

Section 2 reviews prior work. Sections 3 and 4 describe the method and implementation. Section 5 presents the evaluation, and Section 6 concludes.

2 Related Work

2.1 Knowledge Graph Construction and Quality-Aware Extraction

Traditional Knowledge Graph (KG) construction follows a systematic pipeline comprising information extraction, entity linking, relation mapping, and graph assembly (Ji et al. 2022). Early systems such as DBpedia (Auer et al. 2007), YAGO (Suchanek et al. 2007), and Freebase (Bollacker et al. 2008) extract facts from structured and semi-structured sources. During the construction process, quality-aware mechanisms are actively employed to ensure data reliability. These mechanisms include confidence-based filtering, source voting, and entity disambiguation (Dong et al. 2014). Probabilistic knowledge fusion techniques integrate multiple extraction signals to assign reliability scores to generated triples, establishing a foundational layer of quality control directly within the extraction pipeline (Dong et al. 2014).

Embedding-based models map entities and relations into continuous vector spaces for link prediction and downstream inference (Bordes et al. 2013; Wang et al. 2017). Graph neural networks aggregate relational neighborhood information (Schlichtkrull et al. 2018). Surveys and position papers describe how LLMs can support KG construction, completion, and knowledge fusion (Pan et al. 2023, 2024; Bian 2025). General prompting methods include chain-of-thought reasoning (Wei et al. 2022) and reusable prompt patterns (White et al. 2023); retrieval-augmented generation provides a mechanism for conditioning generation on external evidence (Lewis et al. 2020). These general methods should be distinguished from direct evaluations of Text-to-KG extraction. Public systems such as Neo4j Labs’ LLM Knowledge Graph Builder expose a document-to-graph application with configurable schemas and an open-source extraction stack (Neo4j Labs 2024). We include its reproducible core extractor as an external Text-to-KG baseline in Section 5.9.

2.2 Knowledge Graph Quality Assessment

Knowledge graph quality assessment characterizes graph reliability across multiple dimensions, including accuracy, completeness, consistency, and timeliness (Zaveri et al. 2016; Paulheim 2017). Automated assessment methods evaluate these dimensions using diverse computational techniques. Statistical anomaly detection models value distributions to identify irregularities in numerical and literal data (Wienand and Paulheim 2014; Paulheim and Bizer 2014). Embedding-based models evaluate triple plausibility by calculating distances in continuous vector spaces, providing a global semantic perspective on graph integrity (Nickel et al. 2016). Rule-based validation frameworks verify logical structures and schema compliance using formal constraint languages, ensuring explicit ontological consistency (Kontokostas et al. 2014). These methodologies provide comprehensive metrics for evaluating graph quality from local structural to global semantic perspectives.

Lin et al. introduce violation-inducing operations to construct controlled SHACL repair tasks and compare LLM-based repair strategies (Lin et al. 2025). Their results highlight the value of relevant constraints and local graph context in repair prompts.

Crowdsourcing complements automated assessment for ambiguous quality judgments (Acosta et al. 2013). Broader surveys organize quality control around multiple dimensions and stages of KG construction and refinement (Wang et al. 2021; Paulheim 2017). Our diagnostic report records executable structural and source-support checks within a single document graph.

2.3 Knowledge Graph Quality Enhancement and Tool Orchestration

KG enhancement techniques address missing facts and erroneous assertions. Completion methods based on relational paths (Lao et al. 2011), embeddings (Bordes et al. 2013; Yang et al. 2015; Shi and Weninger 2017), and neural architectures (Dettmers et al. 2018) predict missing links. MINERVA learns a policy for navigating a KG to answer queries (Das et al. 2018), while MetaR addresses few-shot link prediction (Chen et al. 2019). These tasks provide mechanisms for inference, but are distinct from evaluating whether an edit preserves valid facts in a corrupted graph. AMIE mines positive logical rules (Galárraga et al. 2013), and RuDiK discovers positive and negative rules (Ortona et al. 2018). Other approaches incorporate logical rules or simple constraints into embedding objectives (Guo et al. 2016; Ding et al. 2018).

Neural and probabilistic approaches further combine logical structure with learned predictions. RNNLogic jointly trains a rule generator and a reasoning predictor through an EM-based procedure (Qu et al. 2021). NBFNet learns path representations for link prediction through a generalized Bellman–Ford formulation (Zhu et al. 2021). pLogicNet combines first-order rules in a Markov logic network with embedding-based variational inference (Qu and Tang 2019). These methods motivate the use of structural priors alongside learned predictions. Our evaluated pipeline normalizes document-field triples and validates generated candidates. We compare candidate filtering with a sequential selector that scores trial graphs.

LLM-based KG repair has been evaluated using explicit SHACL violations and alternative prompting strategies (Lin et al. 2025). More generally, ReAct demonstrates how language-model reasoning can be interleaved with actions to gather information from external sources (Yao et al. 2023). This interaction pattern informs our ReAct-style baseline. Our study isolates diagnostic prompting and source-grounded candidate filtering using shared instructions and frozen candidate responses. A five-part SHACL-context baseline adapts Lin et al.’s design to the same document-field task and model budget. Shared prompts and response-paired scoring separate the roles of context and filtering.

3 Problem Setting and Repair Method

We study source-grounded repair of a small document-level knowledge graph. The input consists of a multiset of triples G , its source text X , a declared document node h_0 , and a permitted relation vocabulary $\mathcal{R}_X$. The graph is field-oriented: each permitted relation can have at most one nonempty value for the document. This setting covers tasks such as document metadata and receipt-field extraction. The reference graph G^* is used only for scoring.

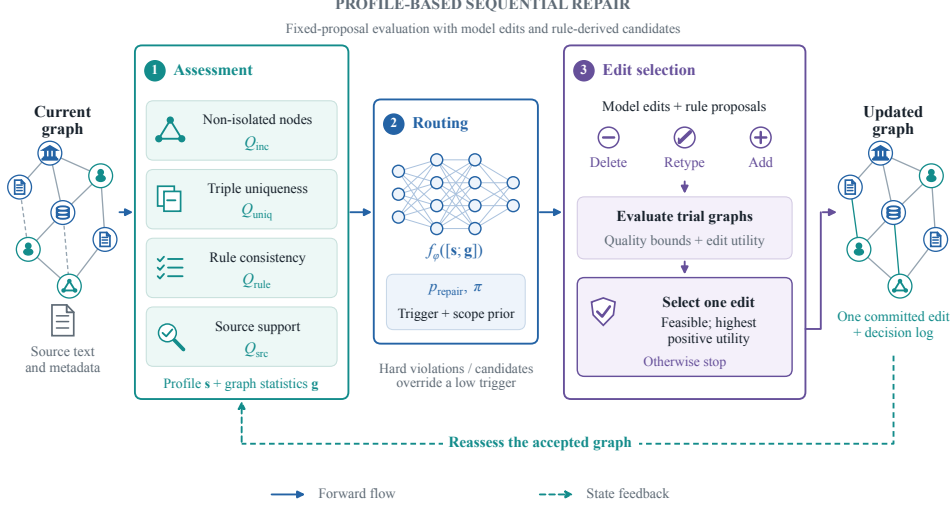


Fig. 1 Profile-based repair architecture. Assessment produces a quality profile; the routing policy supplies a repair trigger and a scope prior. Model edit bundles and rule proposals pass through trial-state checks; each round commits one feasible edit with the highest positive utility. The sequential comparison is described in Section 4.4.

We use literal source matching: a value is supported when it appears in X . Relation correctness and acceptable paraphrases are assessed during evaluation.

Figure 1 shows the repository’s profile-based architecture, which connects assessment, routing, and candidate editing. We first describe the one-call repair pipeline used in the main comparisons. Section 4.4 describes the sequential variant and its evaluation on fixed proposals.

3.1 Structural Normalization and Diagnostic Context

Figure 2 groups diagnostic information into local, graph, and source scopes. For document-field repair, we implement these checks through duplicate counts, the declared head and relation schema, and source matching.

A deterministic preprocessor P removes exact duplicates, corrects an edge when its object equals h_0 but its subject does not, and removes triples whose head or relation violates the supplied schema. Let $G_s = P(G; h_0, \mathcal{R}_X)$.

For a triple $t = (h, r, o)$, define the executable predicates

$$H(t) = \mathbf{1}[h = h_0], \quad R(t) = \mathbf{1}[r \in \mathcal{R}_X], \quad S(t; X) = \mathbf{1}[o \neq \emptyset \wedge o \text{ occurs in } X]. \quad (1)$$

The diagnostic context records duplicate multiplicities, the indices failing H , R , or S , and the relations observed in G_s :

$$D(G_s, X) = (n_{\text{dup}}, I_H, I_R, I_S, \mathcal{R}(G_s)). \quad (2)$$

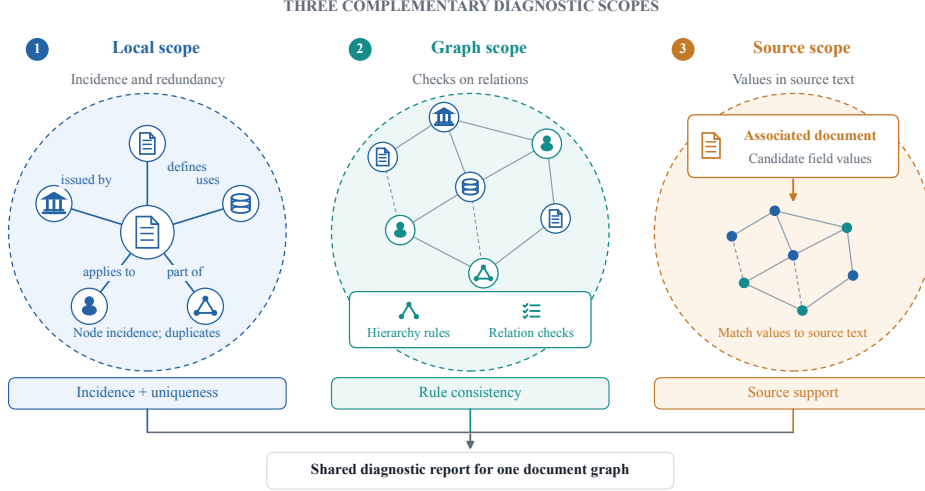


Fig. 2 Three diagnostic scopes. Local checks concern incidence and duplicate facts, graph checks concern schema and relation constraints, and source checks match values to the associated document. The scopes contribute to a shared report. Hierarchy checks use explicit rules. The nodes and relations are schematic; the document-field checks are specified in Eq. (1).

For example, $I_S = \{i : S(t_i; X) = 0\}$. The report is computed from the normalized input. Some entries are empty because preprocessing has already removed the corresponding defect. The matched experiment tests the effect of adding this report to the prompt.

Allowed fields are optional. A graph can pass all visible checks while omitting a fact from the source, so the main pipeline makes one model call for every document.

3.2 Source-Grounded Candidate Generation

A language model generates an ordered list of proposed triples

$$C = L_\theta(X, h_0, \mathcal{R}_X, G_s, D(G_s, X)). \quad (3)$$

The model is instructed to return a complete graph, copy source spans, retain correct fields, and recover missing or corrupted values. A fixed JSON parser turns malformed responses into empty outputs; failures remain in the end-to-end denominator. The base setting omits the diagnostic report. Instructions, source, structural input, model, temperature, and output-token cap remain identical.

3.3 Constraint-Guided Candidate Selection

From the generated candidates, the gate admits only triples satisfying Eq. (1) and retains at most one triple per relation. Writing $\text{uniq}(C)$ for exact deduplication in

generation order, let

$$C_X = \{t \in \text{uniq}(C) : H(t)R(t)S(t; X) = 1\}. \quad (4)$$

The admissible output family is

$$\mathcal{F}(C_X) = \left\{ A \subseteq C_X : \sum_{t \in A} \mathbf{1}[r_t = r] \leq 1, \forall r \in \mathcal{R}_X \right\}. \quad (5)$$

The gate scans candidates in order and retains the first eligible candidate for each relation. The resulting $\hat{G}$ satisfies

$$|\hat{G}| = \max_{A \in \mathcal{F}(C_X)} |A| = |\{r : \exists t \in C_X, r_t = r\}|. \quad (6)$$

Each relation contributes one independent group with capacity one, which gives Eq. (6). The filter therefore retains the largest feasible number of supplied candidates. When several values qualify, it uses generation order to break the tie. We measure factual accuracy and preservation of input facts separately.

3.4 Field Definitions and Evidence Lines

An allowed relation name may leave its intended value unclear. For example, a receipt can contain a trading brand, a registered company, a subtotal, and a final payable amount. In the receipt follow-up, each field has a short definition shared by all repair methods. All methods also receive the same numbered source lines.

The evidence-index variant adds line anchors for each field. Fixed patterns locate company-name terms, address terms and postcodes, date expressions, and total-amount labels. The index includes each anchor, one preceding line, and two following lines. It gives the model source locations to reconsider alongside the current value. The simple baseline receives the same source and definitions, with the index omitted.

We also make whitespace handling consistent with candidate parsing. Let $N(s)$ replace each run of whitespace by one space and trim the ends. The follow-up filter uses

$$S_{\text{ws}}(t; X) = \mathbf{1}[|N(o)| > 0 \wedge N(o) \text{ occurs in } N(X)]. \quad (7)$$

Word boundaries, case, and punctuation are preserved. This predicate replaces S in the follow-up comparisons; the earlier experiments retain their original literal check.

4 Implementation and Experimental Comparisons

4.1 Repair Procedure

Algorithm 1 describes Diagnosis + Gate. The diagnostic report and filter are deterministic; the language model proposes field values.

The runner passes the source, document node, allowed relations, and input triples through a dedicated input object. References are loaded by the scorer. Tests verify

Algorithm 1 Source-grounded document-graph repair

Require: Input multiset G , source X , document node h_0 , relation vocabulary $\mathcal{R}_X$

Ensure: Repaired graph $\hat{G}$ and rejection log J

```
1:  $G_s \leftarrow P(G; h_0, \mathcal{R}_X)$ 
2:  $D \leftarrow D(G_s, X)$ 
3:  $C \leftarrow \text{PARSE}(L_\theta(X, h_0, \mathcal{R}_X, G_s, D))$ 
4:  $\hat{G} \leftarrow \emptyset; J \leftarrow \emptyset; B \leftarrow \emptyset$ 
5: for each candidate  $t = (h, r, o)$  in generation order do
6:   if  $t$  appeared earlier in  $C$  then
7:     append  $(t, \text{duplicate})$  to  $J$ 
8:   else if  $H(t)R(t)S(t; X) = 1$  and  $r \notin B$  then
9:      $\hat{G} \leftarrow \hat{G} \cup \{t\}; B \leftarrow B \cup \{r\}$ 
10:  else
11:    append  $(t, \text{failed check})$  to  $J$ 
12:  end if
13: end for
14: return  $\hat{G}, J$ 
```

that changing reference values and defect labels leaves model inputs unchanged. The document node is task metadata; the legacy data loader reads its identifier from the reference record.

With hash-based schema and duplicate checks, preprocessing takes expected $O(|G|)$ triple operations. Filtering k candidates costs $O(k(1+c_X))$, where c_X is the cost of literal source matching, and uses $O(k + |\mathcal{R}_X|)$ auxiliary storage. String operations depend on text length; model inference is measured separately.

4.2 Matched Context and Filtering Comparisons

We use one system prompt with three context settings: base input, base plus diagnostics, and base plus SHACL validation. Scoring each response before and after filtering gives six configurations from three model calls per input. The raw/gated pair shares exactly the same candidates.

The matched experiment uses `google/gemma-4-26B-A4B-it`, temperature zero, and a 4,000-token completion cap. Call counts and completion caps are the same across settings. We report input tokens separately because the contexts differ in length. Request times include recorded retries and pacing waits. Each completed outcome is stored; malformed responses remain in the evaluation. The earlier Claude comparison uses method-specific prompts and measures whole-pipeline differences.

4.3 SHACL-Context Baseline

Lin et al. use five prompt components: a primer, violation context, SHACL manifest, graph context, and instructions (Lin et al. 2025). We adapt their full-manifest/full-graph ($M+G$) configuration to document-field repair. The pySHACL validator checks

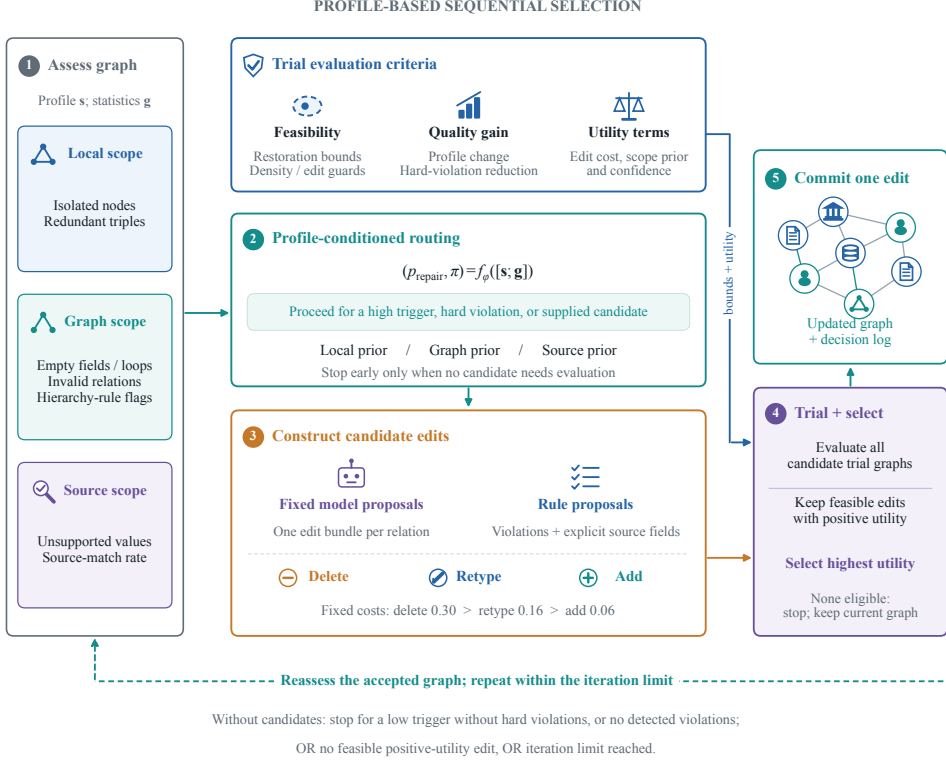


Fig. 3 Sequential selection procedure. The quality profile conditions routing, with hard-violation and supplied-candidate overrides for a low repair probability. Each round selects one feasible edit with the highest positive utility and reassesses the accepted graph. Candidate edits combine fixed model proposal bundles and rules derived from detected violations.

the document head, allowed predicates, nonempty values, and maximum field counts. A SPARQL constraint checks whether values occur in the source text.

The adapted baseline returns complete JSON graphs, includes the source text, and groups a document’s violations into one call. These choices match our task and call budget. We call it *Lin-style SHACL context (adapted)*. RDF stores unique triples; the shared JSON input also preserves duplicate occurrences. We score both raw and filtered outputs for every context setting.

4.4 Sequential Selection Comparison

Figure 3 expands the routing and editing stages of Figure 1. Its profile contains the percentage of non-isolated nodes (Q_{inc}), triple uniqueness (Q_{uniq}), consistency with implemented rules (Q_{rule}), and source support (Q_{src}). The routing network takes these scores s and graph statistics g (node count, triple count, density, and violation count), and returns a repair probability p_{repair} and a scope prior π . The selector combines this prior with trial-state gain, operation cost, and feasibility checks.

A hard violation or supplied candidate keeps evaluation active even when $p_{\text{repair}} < \tau_{\text{repair}}$. Without candidates, the procedure stops if no violations are detected. Otherwise, it evaluates candidate trial graphs and commits the feasible edit with the highest positive utility. If no such edit exists, it retains the current graph and stops. After an accepted edit, it recomputes the profile and routing outputs, up to the iteration limit.

We also evaluate the repository’s sequential selector on fixed model proposals. This alternative uses a four-score quality profile, a learned scope prior, and a utility score to choose an edit at each step. Each proposed edit bundle is applied to a trial graph, which is checked against the restoration bounds before acceptance. Differences between the normalized input and candidate graph define one edit bundle per relation. Proposal confidence is 0.7; the horizon is 12 steps. We compare the learned prior with an always-on uniform prior using the default utility coefficients.

The trace records every trial graph, quality profile, utility, decision, and stopping reason. Source matching removes whitespace from both the source and candidate value. The archive also retains traces from the earlier asymmetric normalization for reproducibility. This selector is evaluated separately from Algorithm 1.

4.5 A Shared Profile for Training and Execution

The revised selector uses one feature function for dataset construction and execution. Its eight inputs are the four profile scores, node count, triple count, directed simple-projection density, and the number of detected violations. Labels describe injected defects; they do not enter this feature function. The document identifier and allowed relations are public task metadata. The checkpoint and scaler carry a feature-version identifier; incompatible weights cause an explicit uniform-prior fallback.

The source-field check recognizes a nonempty value after an explicit allowed field label at a document, line, or sentence boundary. A missing field is reported only when such a value is present in the source. Conflicting values under a repeated label cause abstention. A relation vocabulary alone does not make every relation mandatory. These source-field reports enter the violation count and candidate pool. They do not change the meaning of node incidence: a partial star graph can still have incidence score 100.

Hierarchy checks distinguish downward relations (management and supervision) from upward membership. Ambiguous relation meanings are left unchecked. Endpoint membership is checked against declared entities only for endpoints explicitly typed as entity references; literal field values are not dangling entities. The empty-graph guard applies to both individual and bundled edits. A supplied candidate bypasses the low-probability and no-violation early stops, so each proposed change receives a trial evaluation.

For profile quality $Q \in [0, 100]$, hard-violation count h , operation cost $c(a)$, scope $s(a)$, and proposal confidence $z(a)$, the default utility is

$$U(a) = \frac{Q(G_a) - Q(G)}{100} + 0.20 \frac{\max(0, h(G) - h(G_a))}{\max(1, h(G))} - 0.35c(a) + 0.05 \log(\max(\pi_{s(a)}, 0.05) + 10^{-6}) + 0.02z(a). \quad (8)$$

Here G_a is the trial graph. Delete, retype, and add cost 0.30, 0.16, and 0.06; bundle costs sum their operations. A feasible edit is accepted only if its utility is positive and maximal among the evaluated edits. The four quality terms retain equal weights. Feasibility preserves each satisfied lower bound, does not worsen a deficient dimension, limits a checked component’s regression to two points, and enforces the density bound. These conditions are local constraints, rather than a monotonicity result for reference F1. Pairwise redundancy assessment costs $O(m^2)$ for m triples; T rounds with at most K trial candidates cost $O(TKm^2)$, apart from source-string operations. This differs from the main filter’s linear candidate scan.

We predefine a second utility by subtracting $0.05 \log(1/3 + 10^{-6})$ from the default expression. This makes the prior term zero under a uniform distribution. It leaves the within-step ordering unchanged for a fixed router, but changes which candidates have positive utility. Neither formula is tuned on the archived proposals.

5 Evaluation

We evaluate repair on controlled defects, actual extraction outputs, and external receipt transcripts. We compare full pipelines, then use matched prompts and shared responses to measure the effects of context and filtering.

5.1 Experimental Protocol

Source records and controlled pairs.

The source corpora contain 4,728 government-affairs records from the Shaanxi Provincial Government website, 1,000 financial-regulation records from the National Financial Regulatory Administration, and 500 environmental-regulation records from the Ministry of Ecology and Environment. We sample up to 500 documents per domain and construct a small field-level KG for each document. The evidence string is constructed by serializing the nonempty structured fields as “field name: value, joining them with Chinese sentence separators, and truncating each field value to at most 320 characters (the service label to 100). The same normalized field values define the reference triples. Documents are assigned by source identity to train, validation, and test groups with a 70/15/15 split and seed 42 before corruption, preventing clean/corrupted variants from crossing partitions.

Each controlled instance receives exactly two effective changes selected from six classes: missing triple, duplicate triple, invalid relation, reversed edge, unsupported value, and hierarchy conflict. The manifest records every clean and corrupted triple. The held-out split contains 225 documents, balanced at 75 per domain, and 450 manifested defects (Table 1). The defect types and counts are fixed by this controlled protocol.

Natural extraction errors.

For the same 225 held-out sources, a fresh Claude Haiku call receives only the source text, required document node, and relation vocabulary and constructs a graph from scratch. The structured fields provide silver references for scoring. These inputs contain errors from the extraction call. Ten malformed JSON responses are retained as

Table 1 Controlled paired benchmark. Counts in the last three columns are documents.

Domain	Documents	Defects	Train	Val.	Test
Government	499	998	349	75	75
Finance	500	1,000	350	75	75
Environment	500	1,000	350	75	75
Total	1,499	2,998	1,049	225	225

empty end-to-end outputs. Overall parse success is 95.56%; 77 of 225 output graphs differ from the reference, with 302 multiset triple discrepancies. Independent human review of a frozen discrepancy sample is pending.

Compared repair methods.

All methods receive the same input graph, source evidence, required document node, and relation vocabulary.

- **Source-field Copy** parses declared field-name delimiters in the supplied evidence and reconstructs their values without a model call.
- **No Repair** returns the input unchanged.
- **Rule Only** deduplicates, reverses an edge when its object is the required document node, and removes invalid heads or relations.
- **SHACL-style Repair** applies canonical-head and allowed-predicate shapes (Knublauch and Kontokostas 2017); it cannot reconstruct missing values from text.
- **Direct LLM** makes one source-grounded call and returns a complete graph.
- **ReAct-style Agent** makes a diagnosis call followed by an action call (Yao et al. 2023).
- **Simple Pipeline** applies the same deterministic preprocessing as the diagnosis-plus-gate pipeline, makes one source-grounded call, and only deduplicates the response. It has no diagnostic report or candidate gate. The sequential selector is evaluated separately.
- **Diagnosis + Gate** uses diagnostic-context completion and rejects invalid-head, invalid-relation, unsupported, duplicate, and relation-cardinality proposals.

Unless stated otherwise, model-backed methods use the same served `claude-haiku-4-5-20251001` checkpoint (Anthropic 2025), temperature zero, and endpoint. Failures remain in the denominator. On the controlled set, Direct LLM and Simple Pipeline each have one malformed JSON response; the Diagnosis + Gate and ReAct have none.

Metrics and statistics.

On the controlled benchmark, defect repair requires restoration of the clean triple multiplicity and removal of its corrupted form. We also report clean-fact preservation, over-repair, triple F1, exact multiset graph match, calls, and latency. For a naturally extracted input G_0 , output G_1 , and reference G^* , let d be multiset

Table 2 Controlled held-out results over 225 documents and 450 defects. Higher is better except for over-repair, calls, and latency.

Method	Defect repair	Preservation	Over-repair	Triple F1	Exact match	Calls/doc	Latency/doc
Source-field Copy	1.0000	1.0000	0.0000	1.0000	1.0000	0.00	< 0.001 s
No Repair	0.0000	1.0000	0.0000	0.7615	0.0000	0.00	< 0.001 s
Rule Only	0.3200	1.0000	0.0000	0.8472	0.0622	0.00	< 0.001 s
SHACL-style Repair	0.0000	1.0000	0.0000	0.7998	0.0000	0.00	< 0.001 s
Direct LLM	0.9244	0.9687	0.0758	0.9520	0.8267	1.00	7.25 s
ReAct-style Agent	0.8911	0.9324	0.1667	0.9213	0.5956	2.00	20.50 s
Simple Pipeline	0.9511	0.9723	0.0705	0.9634	0.8400	1.00	9.27 s
Diagnosis + Gate	0.9800	0.9932	0.0167	0.9917	0.9333	1.00	11.44 s

symmetric-difference size. Natural-error reduction is

$$1 - \frac{d(G_1, G^*)}{d(G_0, G^*)},$$

averaged over the 77 non-exact inputs; negative values mean that repair adds more errors than it removes. Confidence intervals use 10,000 paired bootstrap resamples for the new comparisons. Defect outcomes and exact graph matches use exact McNemar tests as descriptive historical comparisons; the natural error-count comparison additionally uses a one-sided paired Wilcoxon test. For the matched comparisons, paired bootstrap and two-sided sign-randomization tests resample whole source documents (10,000 draws, seed 42), keeping the two controlled defects of each document together. Mechanism comparisons are reported as exploratory comparisons without a multiple-testing adjustment.

5.2 Controlled Repair Results

The source-format baseline changes the interpretation of the main result: Source-field Copy obtains 100% repair and exact match with zero model calls. The source lists the reference fields explicitly, allowing the parser to recover them directly. Table 2 shows that Diagnosis + Gate repairs 98.00% of defects, preserves 99.32% of clean facts, and reaches 99.17% triple F1. The strong Simple Pipeline reaches 95.11% repair and 96.34% F1 under the same one-call budget. The Diagnosis + Gate uniquely repairs 15 defects missed by the Simple Pipeline, while the reverse occurs twice (exact McNemar $p = 0.00235$). A document-paired reanalysis gives a repair-rate gain of 2.89 points (95% CI: 1.11–4.89) and a two-sided randomization $p = 0.0067$; the corresponding F1 gain is 2.84 points (95% CI: 1.65–4.25). The Simple Pipeline also improves over Direct LLM, which reaches 92.44% repair.

The Diagnosis + Gate succeeds on 28 defects missed by Direct LLM versus three in the opposite direction ($p = 4.65 \times 10^{-6}$), and on 42 missed by ReAct versus two in the opposite direction ($p = 1.13 \times 10^{-10}$). These tests concern the controlled defect distribution.

5.3 Naturally Occurring Extraction Errors

Table 3 evaluates actual extraction errors on the serialized-record inputs. Source-field Copy again reconstructs all reference graphs. Among the model-based methods, the

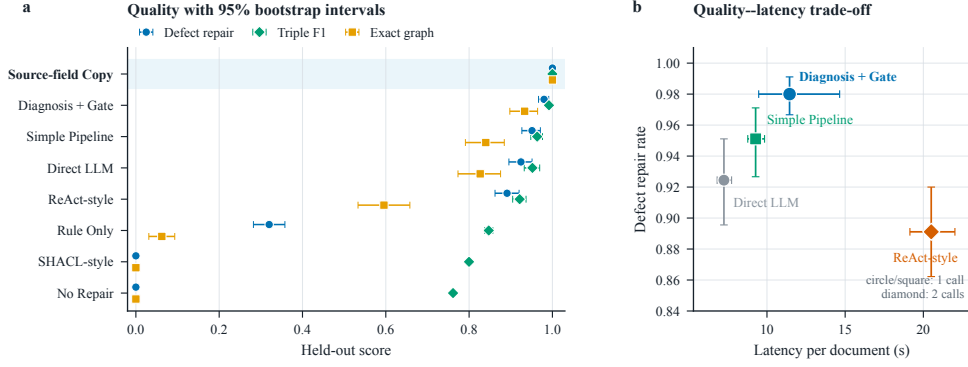


Fig. 4 Controlled repair results. **a**, Defect repair, triple F1, and exact graph match with 95% document-bootstrap intervals. **b**, Quality–latency trade-off for one- and two-call model-backed methods.

Table 3 Repair of graphs produced by an actual extraction call. Error reduction is averaged over the 77 non-exact inputs; other quality metrics use all 225 documents.

Method	Parse	Docs with errors	Error reduction	Preservation	Over-repair	Triple F1	Exact match
Source-field Copy	1.0000	77	1.0000	1.0000	0.0000	1.0000	1.0000
Extracted graph, no repair	0.9556	77	0.0000	1.0000	0.0000	0.8799	0.6578
Simple Pipeline	1.0000	77	−0.5915	0.9971	0.2389	0.8807	0.6711
Diagnosis + Gate	0.9956	77	0.2775	1.0000	0.0292	0.9248	0.6844

unrepaired extraction output has 87.99% F1 and 65.78% exact match. The Simple Pipeline changes few exact-match outcomes, but its mean error reduction on the 77 imperfect inputs is −59.15%: aggressive completion introduces more reference disagreements than it removes. In contrast, the Diagnosis + Gate reduces errors by 27.75%, preserves every initially correct fact on average, and raises F1 to 92.48%. Its paired F1 advantage over the Simple Pipeline is 4.40 points (95% CI: 3.07–5.77), and it prevents 1.03 errors per document on average (95% CI: 0.77–1.32; one-sided Wilcoxon $p = 3.18 \times 10^{-12}$). Exact-match discordance is only three versus zero ($p = 0.25$); triple-level gains are larger than gains in complete graph recovery.

Separating format failures from factual discrepancies.

The 215 successfully parsed extraction outputs have 92.09% initial F1, compared with 93.53% after Diagnosis + Gate and 88.85% after Simple Pipeline. Among their 67 non-exact inputs, mean error reduction is 24.00%; their F1 rises from 74.61% to 79.22%. All 148 initially exact parsed graphs remain exact under Diagnosis + Gate. The ten malformed inputs recover 69.91% F1 but none becomes fully exact. Repair improves valid graphs as well as malformed outputs, with larger gains in triple F1 than in exact graph recovery.

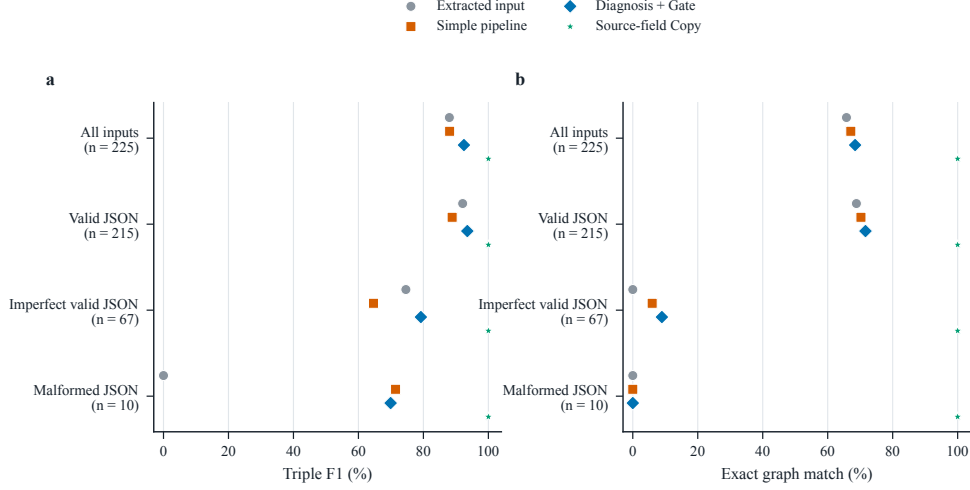


Fig. 5 Repair results by input status. The valid-JSON subset separates factual repair from recovery after parsing failure. References come from the structured source records.

Table 4 Controlled pipeline settings. The Direct LLM row also changes the prompt.

Variant	Repair	Over-repair	F1	Exact
No LLM completion	0.3200	0.0000	0.8472	0.0622
Direct LLM (separate prompt)	0.9244	0.0758	0.9520	0.8267
No constraint gate	0.9800	0.0227	0.9895	0.9333
Diagnosis + Gate	0.9800	0.0167	0.9917	0.9333

5.4 Component and Constraint-Gate Analysis

Table 4 compares the stored component settings. Without LLM completion, the output is the preprocessed graph. Direct LLM uses its own prompt and skips pre-processing. The gate comparison scores the same proposal before and after filtering. Direct LLM is a pipeline comparison, not an isolated preprocessing switch. The next subsection separates prompt context from the other pipeline choices; Section 5.6 tests individual filter and optimizer components.

LLM completion accounts for most recovered fields. Diagnosis + Gate reaches 99.17% F1, compared with 95.20% for Direct LLM. On the shared proposals, filtering leaves defect repair unchanged and reduces over-repair by 0.60 points. It removes eight unsupported triples, all absent from the reference graphs.

5.5 Matched Comparisons and Sequential Selection

We first compare the sequential selector with direct candidate filtering, using the same stored proposals. With either the learned or uniform prior, the selector accepts no further edits on the controlled inputs. Repair stays at 32.00% and F1 at 84.72%, the

Table 5 Matched Gemma comparison on the fixed 60 documents per input condition. Each raw/gated pair shares one response.

Context	Output	Ctrl. repair	Ctrl. F1	Natural F1	Natural exact
Base	Raw	0.9750	0.9951	0.9185	0.6667
Base	Gate	0.9750	0.9962	0.9252	0.6667
Diagnostic	Raw	0.9250	0.9850	0.9079	0.6667
Diagnostic	Gate	0.9250	0.9861	0.9147	0.6667
SHACL context	Raw	0.9500	0.9912	0.9322	0.7000
SHACL context	Gate	0.9500	0.9912	0.9364	0.7000

Table 6 Final-run request costs. Means include retries and rate-pacing waits; source preparation and executor queue time are excluded.

Input	Context	Calls	Input tokens	Output tokens	Wall (s)
Controlled	Base	1.00	947	517	8.58
Controlled	Diagnostic	1.00	1017	512	8.61
Controlled	SHACL	1.00	3080	513	8.45
Natural	Base	1.00	832	458	7.51
Natural	Diagnostic	1.00	901	462	7.90
Natural	SHACL	1.00	2822	455	8.16

Table 7 Input-derived diagnostic scaling. Memory is peak new batch allocation in MiB; source texts are shared.

Triples	Documents	Full (ms)	Local (ms)	Allocation (MiB)
1,000	143	1.20	0.009	0.29
5,000	716	6.08	0.008	1.47
10,000	1,434	13.90	0.008	2.96
50,000	7,173	88.10	0.008	14.84

preprocessing result. On natural inputs, it reaches 88.33% F1 and 65.78% exact match. The traces show two causes: a complete-looking quality profile can still omit source fields, and useful additions can receive non-positive utility under the default score.

The Gemma experiment uses a fixed 60-document subset (20 per domain) under both controlled and natural-input conditions. Three context arms share one system prompt, deterministic preprocessing, temperature, and completion cap. Each candidate response is scored before and after the same gate. Table 5 reports all six combinations. Table 6 reports the different input-token counts and request times under the common call budget and completion cap. The SHACL arm follows the documented adaptation in Section 4.3.

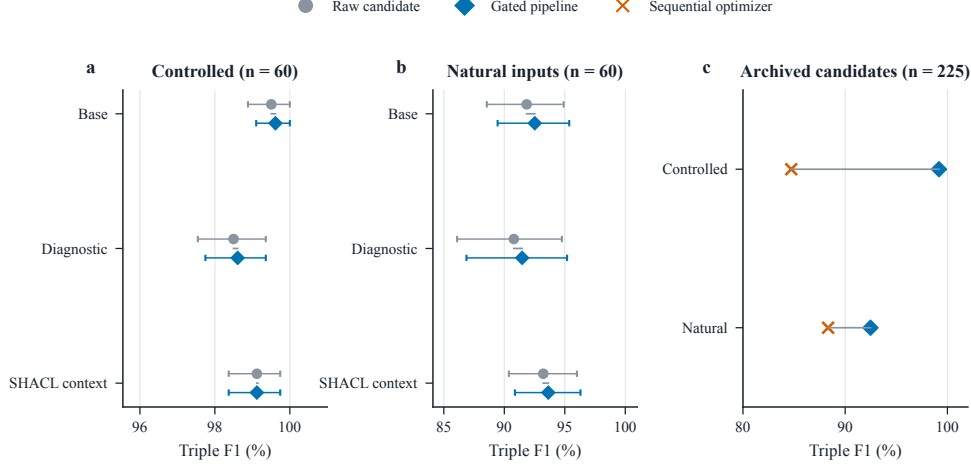


Fig. 6 Context, filtering, and selection comparisons. Panels a–b use Gemma responses on 60 documents; raw and gated outputs share candidates. Error bars are 95% document-bootstrap intervals. Panel c compares filtering and sequential selection on the stored Claude proposals for 225 documents.

After identical filtering, base context reaches 99.62% controlled F1 and 97.50% defect repair, versus 98.61% and 92.50% with diagnostics. The paired F1 difference (diagnostic minus base) is -1.01 points (95% CI: $[-1.67, -0.38]$; exploratory randomization $p = 0.0082$). On natural inputs, base-plus-gate reaches 92.52% F1, diagnostic-plus-gate 91.47%, and the adapted SHACL-context-plus-gate baseline 93.64%. The diagnostic–SHACL difference is -2.16 points (95% CI: $[-5.26, 0.02]$; $p = 0.105$), with the interval including zero.

Filtering has a smaller, response-paired effect. Within diagnostic-context natural outputs, F1 increases from 90.79% to 91.47%, a 0.69-point gain (95% CI: 0.20–1.29; exploratory $p = 0.0335$), without changing exact match. Across the 360 responses, the gate rejects 27 candidate occurrences, all non-reference triples. In this setting, filtering improves candidate quality, while added diagnostic context lowers F1 relative to the base prompt.

We also replay a call policy based on visible head, relation, duplicate, cardinality, and source-support violations. It chooses between the input and stored repair output for each document. On the 450-item natural/clean stream, the reference-free visible-violation policy routes 6.22% of inputs but misses 63.64% of imperfect graphs; its output F1 is 94.46%, compared with 95.82% for always calling repair. The main method calls repair for every document; the visible-check policy misses many missing or incorrectly assigned fields.

5.6 Individual Filter and Optimizer Ablations

We test the filter checks on the 360 Gemma responses from the matched-context study and the 180 responses from the receipt follow-up. Each response is rescored under eight configurations: all checks, each of five checks removed, duplicate and cardinality checks removed together, and all checks removed. These 4,320 scored outputs share 540

Table 8 Paired F1 contribution of filter checks, in percentage points. Each cohort contains 60 documents with three generation arms. Positive values favour the full filter.

Removed check(s)	Controlled	Natural	Receipts
Duplicate	+0.000	+0.000	+0.000
Document head	+0.037	+0.085	+0.000
Relation type	+0.000	+0.000	+0.000
Source support	+0.037	+0.505	+0.060
Cardinality	+0.000	+0.000	+0.000
Duplicate + cardinality	+0.000	+0.000	+0.000
All checks	+0.073	+0.590	+0.060

model responses. Candidate order stays fixed. Joint removal tests whether cardinality masks the duplicate check.

Table 8 reports full-filter F1 minus each modified-filter F1. We average the paired changes over the three generation arms within each document, then over documents. The filter rejects 28 candidate occurrences, all absent from the references: 11 have the wrong document head and 17 fail source support. Other checks remove no additional candidates. On natural inputs, source support contributes 0.505 F1 points and the head check 0.085 points. Their exploratory effects do not pass Holm correction across the 21 cohort/check comparisons. The response-level audit also records any correct fact displaced when a check is removed.

Sequential optimizer components.

The frozen version-1 study replays 13 settings on the same fixed proposals for 225 controlled and 225 natural graphs, giving 5,850 outcomes. Every setting starts from the same preprocessed graph. The reference uses an always-on uniform prior. The learned-prior arm keeps its trigger on; the next arm restores the learned trigger. The remaining arms each change one component of the uniform reference. Table 9 gives scores and accepted action counts.

The local, graph, and source score ablations zero the named quality terms and remove their bounds. Remaining weights keep their original values. Violation detectors, hard-violation reward, candidate generation, and stopping rules remain active. These settings isolate score and bound components. The no-quality-bounds arm removes lower-bound and regression checks while retaining density and empty-graph protection.

On controlled inputs, removing action cost raises F1 from 84.72% to 89.30% and defect repair from 32.00% to 46.89%. It accepts 68 edits; the reference accepts none. Among the reference's 220 candidate trials, 218 have non-positive utility and two fail source-quality checks. In another 86 documents, the optimizer stops because it detects no violation. Thus both the stopping profile and the action score limit field restoration.

On natural inputs, removing cost changes F1 from 88.33% to 88.13%, with 12 accepted edits instead of nine. More edits therefore do not consistently improve reference accuracy. Removing local or source score/bounds reduces F1 to 87.99%. The

Table 9 Executed optimizer ablations with fixed proposals. Accepted counts are edit bundles/actions, shown as controlled / natural.

Setting	Controlled F1	Natural F1	Accepted
Always-on uniform	0.8472	0.8833	0 / 9
Learned prior, always on	0.8472	0.8833	0 / 9
Learned prior and trigger	0.8472	0.8833	0 / 9
No prior penalty	0.8472	0.8833	0 / 9
No action cost	0.8930	0.8813	68 / 12
No density bound	0.8472	0.8833	0 / 9
No quality bounds	0.8472	0.8833	0 / 9
One iteration	0.8472	0.8833	0 / 9
No rule candidates	0.8472	0.8833	0 / 9
Rule candidates only	0.8472	0.8799	0 / 0
No local score/bounds	0.8472	0.8799	0 / 0
No graph score/bound	0.8472	0.8833	0 / 9
No source score/bound	0.8472	0.8799	0 / 9

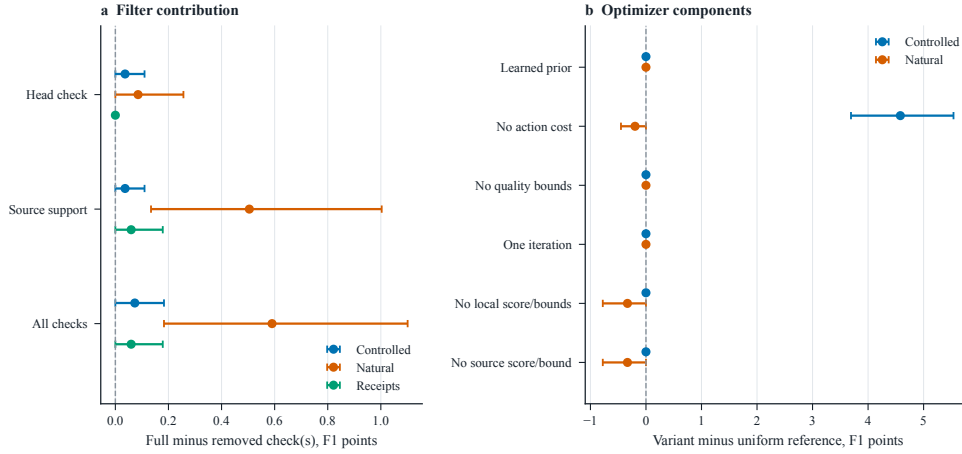


Fig. 7 Completed offline ablations. Left: paired F1 contribution of head, source-support, and all filter checks, averaged within each document across generation arms. Right: F1 changes from the always-on uniform optimizer for selected component removals. Bars are marginal 95% document-bootstrap intervals. All 13 optimizer configurations are reported in Table 9.

learned prior, learned trigger, density bound, quality bounds, and one-step horizon leave output scores unchanged in this replay. The saved neural weights have the training-feature mismatch discussed in the execution audit; these frozen comparisons measure their runtime effect. The version-2 study below uses a retrained shared-feature model. Per-candidate traces retain trial graphs, quality profiles, utilities, constraint decisions, and stop reasons.

Table 10 Version-2 selector on the same archived proposals. Relative priors use the uniform distribution as their zero point. Accepted edits are controlled / natural.

Prior / offset	Controlled F1	Natural F1	Accepted
Uniform / absolute	0.8472	0.8833	0 / 10
Uniform / relative	0.8472	0.8833	0 / 10
Learned / absolute	0.8472	0.8799	0 / 10
Learned / relative	0.8929	0.8764	156 / 51

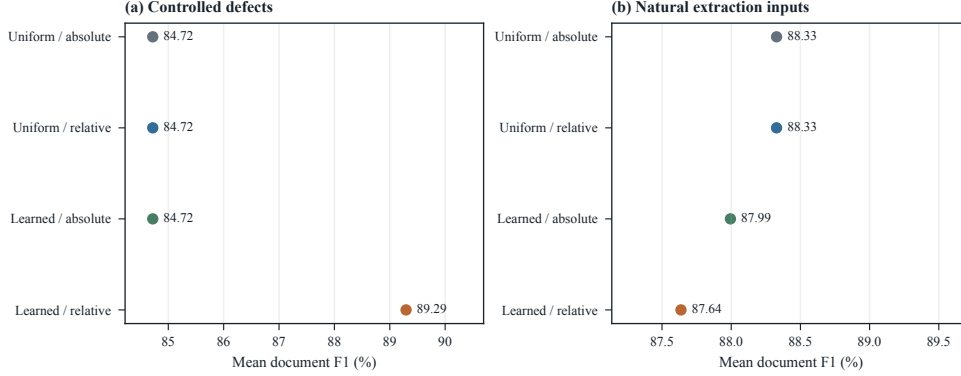


Fig. 8 Revised selector with shared training and execution features. Points show mean document F1 for fixed prior and offset choices on archived proposals.

Shared-feature retraining and revised execution.

The version-2 router retains the $8 \rightarrow 32 \rightarrow 16$ network with one repair head and three scope outputs (884 parameters), seed 42, and the original document-group split. Training fits the scaler only on the 2,098 training rows; validation and test each contain 450 rows. Repair classification and dominant-scope accuracy are 100% on this controlled test split. The repair Brier score is 1.02×10^{-9} . These labels follow the known corruption construction; the result measures recognition of those defects.

We execute four fixed router/utility combinations on the same 225 controlled and 225 natural-input proposals (1,800 outcomes). Table 10 reports the revised implementation separately from the frozen component study above. The revised uniform default evaluates 471 controlled candidate trials, compared with 220 previously, but still accepts no edits. Candidate evaluation alone therefore does not remove the positive-utility barrier. With the retrained prior, the relative offset raises controlled F1 from 84.72% to 89.29%, while natural-input F1 changes from 87.99% to 87.64%. The uniform alternatives give identical outputs. The default absolute offset is retained; the contrast shows that a threshold shift can help one cohort and harm another. The empty-value diagnostic fix changes no archived case. All these proposals were previously evaluated, so this is a fixed-candidate execution comparison. It does not alter the main one-call repair results.

Table 11 Post-hoc selection diagnostics on 225 inputs per condition. The oracle uses reference labels only for analysis. Trial counts can exceed document counts.

Diagnostic	Controlled	Natural
Preprocessed graph F1	0.8472	0.8799
Uniform optimizer F1	0.8472	0.8833
Fixed-proposal oracle F1	0.9964	0.9173
Initially no detected violation	86	198
Imperfect among those graphs	72	50
Improvement available among those graphs	70	8
Improving trials with non-positive utility	212	1

Where useful edits are lost.

We use the stored proposals and trial graphs to locate the selection bottleneck. For each document, we enumerate every subset of its model-proposal bundles and score the resulting graph against the reference. We apply the same edit operations as the runtime but omit its stopping, cost, and feasibility checks. This reference-guided oracle measures the best F1 within those fixed subsets; it is an analysis tool, not a repair method. Detector-generated actions are excluded.

The enumeration covers 2,144 subsets across 450 inputs. Table 11 separates candidate availability from stopping and scoring. On controlled inputs, 86 graphs have no detected initial violation, but 72 still miss reference fields. Of those initial no-violation graphs, 70 already have a proposal subset that improves F1. Among evaluated trials, 212 improve reference F1 but receive non-positive utility. The corresponding natural-input counts are 8 graphs and 1 trial.

Removing action cost improves F1 on 67 controlled documents, leaves it unchanged on 158, and harms 0. On natural inputs it improves 0, leaves 222 unchanged, and harms 3. The three harmed graphs were already imperfect after preprocessing; none of the 148 exact natural inputs loses F1. The paired controlled change is 4.58 points (95% document-bootstrap CI: 3.69–5.54). These document-level results explain why the same cost change has different effects across the two conditions. No parameters are changed using this analysis.

5.7 Cross-Model Robustness

The cross-model comparison uses Direct LLM, Simple Pipeline, and Diagnosis + Gate on a fixed 60-case subset (20 per domain) with Claude Haiku and google/gemma-4-26B-A4B-it (Gemma Team 2026). Within each method, inputs, prompt templates, temperature, and one-call budgets are held fixed across models. Table 12 shows that the Diagnosis + Gate has the highest repair rate with both models. Its absolute F1 advantage over the Simple Pipeline is larger with Claude; Gemma’s stronger direct outputs leave less room for post-processing. This comparison measures whole-pipeline behavior across models. Section 5.5 isolates the effect of diagnostic context.

Table 12 Cross-model controlled results on the same 60 cases.

Model	Method	Repair	F1	Exact
Claude	Direct	0.958	0.970	0.867
Claude	Simple	0.958	0.966	0.833
Claude	Diag.+Gate	0.975	0.991	0.917
Gemma	Direct	0.958	0.988	0.883
Gemma	Simple	0.967	0.984	0.867
Gemma	Diag.+Gate	0.983	0.990	0.917

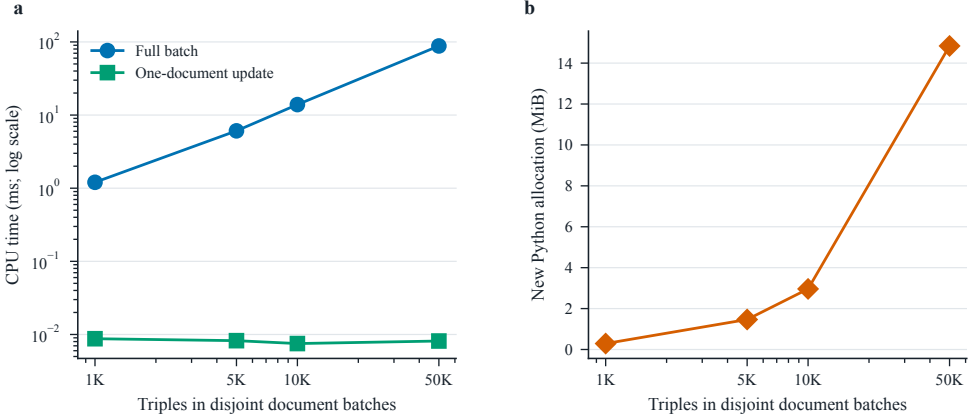


Fig. 9 Scaling of input-derived diagnostics on disjoint document batches. Local updates recompute one document. Memory records new batch allocations with shared source text.

5.8 Diagnostic Batch Scaling

We time the input-derived diagnostic report in Eq. (2) on disjoint batches containing 1K–50K triples. A full pass visits every document; a local update recomputes one changed document. Full times are medians of seven runs and local times of 200 runs. Table 7 reports the measurements. The memory figure is peak new Python allocation for batch construction, with source text and schema objects shared. The measured operations are diagnostic construction and updates; model calls are timed separately.

5.9 Secondary Checks

An independent Gemma judge evaluates 180 source–triple pairs without method identity. Its source-support score correlates with exact reference validity at Pearson $r = 0.794$ and Spearman $\rho = 0.807$ (both $p < 2.4 \times 10^{-40}$). A fixed 50-item subset produces the same mean, 0.96, in five additional runs. This measures repeatability of the model judge.

Table 13 External SROIE results on 60 receipt transcripts. Exact match is over annotated fields. Normalized F1 ignores case and whitespace in values.

Method	Triple F1	Exact match	Normalized F1	Preservation
Extracted input	0.7944	0.3333	0.8153	1.0000
Rule Only	0.7944	0.3333	0.8153	1.0000
Source-field Copy	0.0000	0.0000	0.0000	0.0000
Base, raw	0.7944	0.3333	0.8153	1.0000
Base, gate	0.7915	0.3333	0.8123	0.9944
Diagnostic, raw	0.7944	0.3333	0.8153	1.0000
Diagnostic, gate	0.7915	0.3333	0.8123	0.9944
SHACL context, raw	0.7986	0.3333	0.8194	1.0000
SHACL context, gate	0.7956	0.3333	0.8165	0.9944

We also compare the extraction path with Neo4j Labs’ reproducible `LLMGraphTransformer` core on 45 balanced TNEWS documents (Neo4j Labs 2024). Category accuracy is 0.556 for our schema-guided extractor and 0.533 for Graph Builder; paired outcomes do not differ ($p = 1.00$). TNEWS has no gold triples, so this supports only comparable behavior under that extraction protocol.

5.10 External Receipt Text

We use SROIE receipt transcriptions and human key-information labels (Huang et al. 2019) to test repair outside the Chinese record collections. A fixed random sample contains 60 of the 626 documents in the community-corrected trainval mirror (seed 20260922). Text lines are joined in their stored order. The model receives this text and the four field names: company, date, address, and total. Field values are read separately by the scorer. One receipt lacks an address annotation, giving 239 evaluated fields; the missing annotation is excluded from scoring for every method. All four fields remain available to the model.

A Gemma call first extracts each graph. The three repair contexts then receive the same extracted graph, using the settings in Section 4.2. Sampling and settings were fixed before inference. This gives 60 extraction calls and 180 repair outcomes, each of which is scored before and after filtering. We reuse the source-copy and structural baselines without receipt-specific tuning. The primary score uses exact triple matching. A secondary score ignores case and whitespace in field values to separate these formatting differences from other errors.

The extracted graphs reach 79.44% F1. After filtering, base, diagnostic, and SHACL contexts reach 79.15%, 79.15%, and 79.56%, respectively (Table 13). The diagnostic–base difference is 0.00 points (95% CI: [0.00, 0.00]; paired randomization $p = 1.0000$). The field-copy parser reaches 0.00% F1 on these transcripts, compared with 100% on the serialized records.

Of the 239 reference values, 220 occur literally in the transcripts. The other 19 values are absent as exact source strings. Across the 180 repair responses, filtering rejects 3 candidate occurrences, all matching a reference value. They concern the same address in three arms: the source contains a double space, while the JSON parser collapses candidate whitespace. A post-run check applies the same whitespace

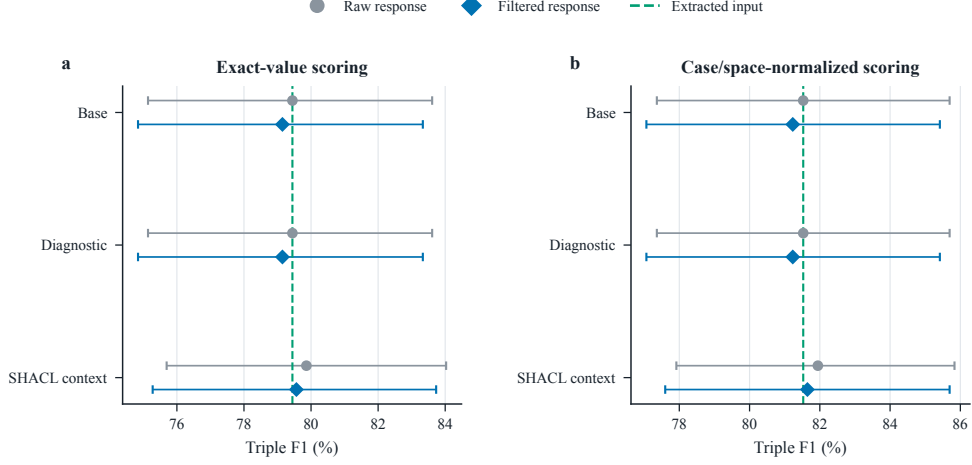


Fig. 10 Repair on independent receipt text. Points show mean triple F1 and 95% document-bootstrap intervals. Raw and gated results share model responses; the dashed line marks the extracted input.

normalization to the source and retains this address in all three arms. With the original responses, F1 becomes 79.44% for base and diagnostic contexts and 79.86% for SHACL context; exact graph match remains 33.33%. The main table keeps the fixed protocol results, and the separate check identifies the effect of this normalization mismatch.

The run contains 240 recorded requests, including retries, and 240 successfully parsed outcomes. Mean repair request times are 7.87, 7.92, and 8.05 seconds for base, diagnostic, and SHACL contexts. Their mean input lengths are 661, 715, and 2276 tokens. These times include request pacing and exclude shared extraction and context preparation.

5.11 Field-Evidence Follow-up on New Documents

The first receipt study suggests that visible schema violations miss many field-selection errors. Base and diagnostic prompts return all 60 input graphs unchanged. Of 239 reference fields, 190 match exactly. The 49 mismatches include 27 case, spacing, punctuation, or amount-format differences, ten span-boundary differences, eight different source values, and four values absent from the normalized source. These categories describe reference mismatches; they separate formatting from other repair targets.

We develop the field-evidence index in Section 3.4 on 20 new receipts, then freeze it before testing 60 further receipts. The original 60, development 20, and test 60 documents have disjoint IDs and no identical normalized transcripts. The sample is fixed with seed 20260923. Test labels are downloaded only after all test predictions are stored. Every arm receives the same field definitions, numbered source lines, input graph, and instructions. Simple uses these inputs directly; Evidence Index adds field anchors; SHACL adds validation context. All three share the whitespace-consistent filter.

Table 14 Frozen follow-up on 60 new receipts. All repair arms share field definitions and source lines. Gate uses consistent whitespace matching. Exact match is over annotated fields.

Method	Triple F1	Exact graph	Normalized F1	Preservation
Extracted input	0.7833	0.3500	0.7958	1.0000
Simple, raw	0.7875	0.2833	0.8083	0.9250
Simple, gate	0.7875	0.2833	0.8083	0.9250
Evidence Index, raw	0.8333	0.4000	0.8625	0.9306
Evidence Index, gate	0.8351	0.4000	0.8643	0.9306
SHACL context, raw	0.7833	0.2833	0.8042	0.9347
SHACL context, gate	0.7833	0.2833	0.8042	0.9347

Development F1 is 80.00% for the extracted graphs, 82.50% for Simple, 85.00% for Evidence Index, and 80.00% for SHACL. We retain one fixed index design and all three arms for the test. The primary comparison, frozen with the code and scoring rules, is Evidence Index minus Simple in document-mean triple F1.

On the new test, extracted graphs reach 78.33% F1. Simple, Evidence Index, and SHACL with filtering reach 78.75%, 83.51%, and 78.33%, respectively (Table 14). Simple changes F1 by 0.42 points relative to the input (95% CI: $[-4.17, 5.00]$). The primary Evidence Index–Simple difference is 4.76 points (95% CI: $[2.50, 7.26]$; two-sided paired randomization $p = 0.0004$). Exact graph match is 28.33%, 40.00%, and 28.33% for the three repair arms.

Simple restores 16 of 52 initially non-exact field values and changes 15 initially exact values to non-exact values. Evidence Index restores 26 of 52 initially non-exact field values and changes 14 initially exact values to non-exact values. SHACL restores 13 of 52 initially non-exact field values and changes 13 initially exact values to non-exact values. The index restores 11 punctuation-only differences and 6 span-boundary differences. Of its lost exact matches, 10 preserve the numeric amount but change its display format. A separate numeric check removes currency prefixes and thousands separators from the total field. Its accuracy is 58/60 for the input, 59/60 for Simple, 59/60 for Evidence Index, and 58/60 for SHACL. This check distinguishes a changed amount from a changed display format.

Filtering rejects 1 candidate occurrence; 0 rejected candidates match a reference value. The rejected address joins source fragments into a span absent from the text. Filtering changes Evidence Index F1 by 0.18 points and leaves the other two arms unchanged.

The three arms change 31, 39, and 28 of the 60 input graphs. Mean repair request times are 7.91, 7.88, and 8.11 seconds, with 1784, 2153, and 3404 input tokens. The run contains 240 actual requests for 60 extraction and 180 repair outcomes. Raw and gated scores share responses; request times include pacing and retries.

5.12 Scope and Limitations

The evaluated graphs describe document fields with one value per relation. Cross-document alignment, causal reasoning, and repair of large connected KGs require different data and constraints. The scaling results concern disjoint document batches and exclude model inference.

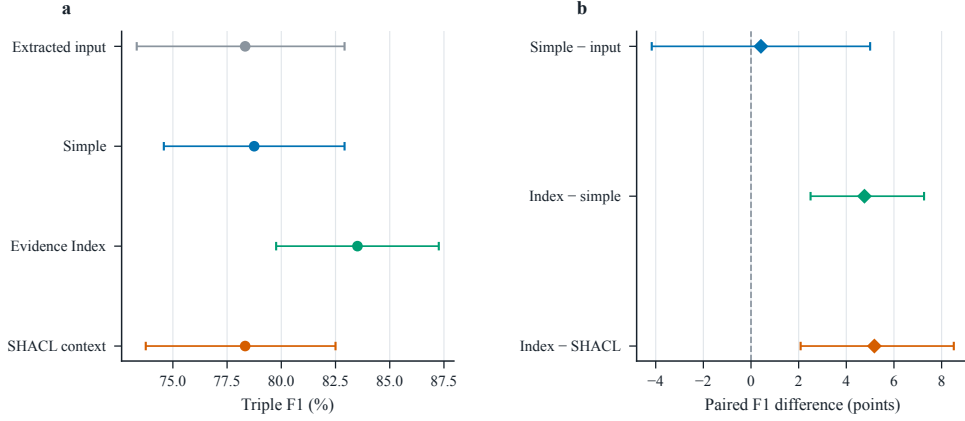


Fig. 11 Follow-up on new receipts. Left: mean triple F1 with 95% document-bootstrap intervals. Right: paired F1 changes; intervals resample whole documents. All repair methods share field definitions and source lines.

The three Chinese collections use field-serialized evidence and silver references. Their perfect source-copy result reflects this source format. SROIE adds independent transcripts and human field labels, with an initial 60-document study and a separate 20-document development set and 60-document test from one English/Malay receipt collection. Its transcripts and field labels sometimes differ in spelling or spacing. These differences affect strict matching and literal source checks. Broader domains and models remain useful tests of transfer.

The filter checks literal occurrence and field counts. It can retain a value under the wrong relation, reject a valid paraphrase, or keep the wrong value when several candidates qualify. Generation can also omit a correct input fact. These cases motivate relation-aware validation and the two independent human-review tasks described below.

5.13 Reproducibility and Availability

Code, prompts, data manifests, predictions, and analysis scripts are available at https://github.com/turambar928/MCP_based_KG_construction. Each experiment records model settings, parsing status, requests, and token usage. Completed responses, including failures, are retained. Data sources, fixed samples, checksums, and commands are listed in each experiment README.

The human-review package contains two independent tasks: 200 input/reference discrepancies and 200 actual system edits. Each annotator receives a separate blinded workbook and offline form, with Chinese instructions and examples. The coordinator merges completed labels, computes agreement before adjudication, and reports uncertain cases separately. The review is pending; the package contains blank labels. The SROIE results use the dataset’s existing human field annotations.

6 Conclusion

We studied document-graph repair through source-based generation and field validation. The method combines structural normalization, optional diagnostic context, one model call, and candidate filtering. On the Chinese record benchmark, Diagnosis + Gate repairs 98.00% of controlled defects and raises F1 on extracted graphs from 87.99% to 92.48%. Direct field copying reaches 100% on that source format.

Independent receipt text reveals different problems: field boundaries, alternative source values, and inconsistent text normalization. We correct whitespace handling and compare shared field definitions with an added source-evidence index on new documents. On the frozen 60-document test, Simple, Evidence Index, and SHACL reach 78.75%, 83.51%, and 78.33% F1, compared with 78.33% for extracted inputs. The paired index contribution is 4.76 points, with a 95% interval from 2.50 to 7.26.

Component replays show that action-cost penalties suppress controlled repairs, while removing them does not improve natural-input F1. In the sampled model responses, filtering acts mainly through source support and document-head checks.

These results distinguish the full schema-aware repair step from the added effects of evidence context and filtering. Remaining work includes independent review of sampled edits, relation-aware validation, and tests on a wider range of document schemas.

References

- Acosta M, Zaveri A, Simperl E, et al (2013) Crowdsourcing linked data quality assessment. In: International Semantic Web Conference, pp 260–276, https://doi.org/10.1007/978-3-642-41338-4_17
- Anthropic (2025) Introducing Claude Haiku 4.5. Anthropic model release announcement, URL <https://www.anthropic.com/news/claude-haiku-4-5>, published 15 October 2025; accessed 17 September 2026
- Auer S, Bizer C, Kobilarov G, et al (2007) DBpedia: A nucleus for a web of open data. In: The Semantic Web: 6th International Semantic Web Conference and 2nd Asian Semantic Web Conference (ISWC/ASWC 2007). Springer, pp 722–735, https://doi.org/10.1007/978-3-540-76298-0_52
- Bian H (2025) LLM-empowered knowledge graph construction: A survey. arXiv preprint, <https://doi.org/10.48550/arXiv.2510.20345>, [arXiv:2510.20345](https://arxiv.org/abs/2510.20345)
- Bollacker K, Evans C, Paritosh P, et al (2008) Freebase: A collaboratively created graph database for structuring human knowledge. In: Proceedings of the 2008 ACM SIGMOD International Conference on Management of Data, pp 1247–1250, <https://doi.org/10.1145/1376616.1376746>
- Bordes A, Usunier N, Garcia-Duran A, et al (2013) Translating embeddings for modeling multi-relational data. In: Advances in Neural Information

- Processing Systems, URL https://papers.nips.cc/paper_files/paper/2013/hash/1cecc7a77928ca8133fa24680a88d2f9-Abstract.html
- Chen M, Zhang W, Zhang W, et al (2019) Meta relational learning for few-shot link prediction in knowledge graphs. In: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP), pp 4217–4226, <https://doi.org/10.18653/v1/D19-1431>
- Das R, Dhuliawala S, Zaheer M, et al (2018) Go for a walk and arrive at the answer: Reasoning over paths in knowledge bases using reinforcement learning. In: International Conference on Learning Representations, URL <https://arxiv.org/abs/1711.05851>
- Dettmers T, Minervini P, Stenetorp P, et al (2018) Convolutional 2D knowledge graph embeddings. In: Proceedings of the AAAI Conference on Artificial Intelligence, <https://doi.org/10.1609/aaai.v32i1.11573>
- Ding B, Wang Q, Wang B, et al (2018) Improving knowledge graph embedding using simple constraints. In: Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics, pp 110–121, <https://doi.org/10.18653/v1/p18-1011>
- Dong X, Gabrilovich E, Heitz G, et al (2014) Knowledge vault: A web-scale approach to probabilistic knowledge fusion. In: Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp 601–610, <https://doi.org/10.1145/2623330.2623623>
- Galárraga LA, Teflioudi C, Hose K, et al (2013) AMIE: Association rule mining under incomplete evidence in ontological knowledge bases. In: Proceedings of the 22nd International Conference on World Wide Web, pp 413–422, <https://doi.org/10.1145/2488388.2488425>
- Gemma Team (2026) Gemma 4 technical report. arXiv preprint, <https://doi.org/10.48550/arXiv.2607.02770>, [arXiv:2607.02770](https://arxiv.org/abs/2607.02770)
- Guo S, Wang Q, Wang L, et al (2016) Jointly embedding knowledge graphs and logical rules. In: Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing, pp 192–202, <https://doi.org/10.18653/v1/d16-1019>
- Huang Z, Chen K, He J, et al (2019) ICDAR2019 competition on scanned receipt OCR and information extraction. In: 2019 International Conference on Document Analysis and Recognition (ICDAR). IEEE, pp 1516–1520, <https://doi.org/10.1109/ICDAR.2019.00244>
- Ji S, Pan S, Cambria E, et al (2022) A survey on knowledge graphs: Representation, acquisition, and applications. IEEE Transactions on Neural Networks and Learning Systems 33(2):494–514. <https://doi.org/10.1109/tnnls.2021.3070843>

- Knublauch H, Kontokostas D (2017) Shapes constraint language (SHACL). W3C Recommendation, URL <https://www.w3.org/TR/2017/REC-shacl-20170720/>
- Kontokostas D, Westphal P, Auer S, et al (2014) Test-driven evaluation of linked data quality. In: Proceedings of the 23rd International Conference on World Wide Web, pp 747–758, <https://doi.org/10.1145/2566486.2568002>
- Lao N, Mitchell T, Cohen WW (2011) Random walk inference and learning in a large scale knowledge base. In: Proceedings of the 2011 Conference on Empirical Methods in Natural Language Processing, pp 529–539, URL <https://aclanthology.org/D11-1049/>
- Lewis P, Perez E, Piktus A, et al (2020) Retrieval-augmented generation for knowledge-intensive NLP tasks. In: Advances in Neural Information Processing Systems, pp 9459–9474, URL https://papers.nips.cc/paper_files/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html
- Lin TW, Fierro G, Li H, et al (2025) Systematic evaluation of knowledge graph repair with large language models. arXiv preprint, <https://doi.org/10.48550/arXiv.2507.22419>, arXiv:2507.22419
- Neo4j Labs (2024) LLM Knowledge Graph Builder. Open-source software repository, URL <https://github.com/neo4j-labs/llm-graph-builder>, version inspected at commit 5ff7af3e9bb9226e1bbeed02f70f8d98697727a7; accessed 16 September 2026
- Nickel M, Murphy K, Tresp V, et al (2016) A review of relational machine learning for knowledge graphs. Proceedings of the IEEE 104(1):11–33. <https://doi.org/10.1109/jproc.2015.2483592>
- Ortona S, Meduri VV, Papotti P (2018) Robust discovery of positive and negative rules in knowledge bases. In: Proceedings of the 34th IEEE International Conference on Data Engineering, pp 1168–1179, <https://doi.org/10.1109/icde.2018.00108>
- Pan JZ, Razniewski S, Kalo JC, et al (2023) Large language models and knowledge graphs: Opportunities and challenges. arXiv preprint, <https://doi.org/10.48550/arXiv.2308.06374>, arXiv:2308.06374
- Pan S, Luo L, Wang Y, et al (2024) Unifying large language models and knowledge graphs: A roadmap. IEEE Transactions on Knowledge and Data Engineering 36(7):3580–3599. <https://doi.org/10.1109/tkde.2024.3352100>
- Paulheim H (2017) Knowledge graph refinement: A survey of approaches and evaluation methods. Semantic Web 8(3):489–508. <https://doi.org/10.3233/sw-160218>
- Paulheim H, Bizer C (2014) Improving the quality of linked data using statistical distributions. International Journal on Semantic Web and Information Systems 10(2):63–86. <https://doi.org/10.4018/ijswis.2014040104>

- Qu M, Tang J (2019) Probabilistic logic neural networks for reasoning. In: Advances in Neural Information Processing Systems, URL https://papers.nips.cc/paper_files/paper/2019/hash/13e5ebb0fa112fe1b31a1067962d74a7-Abstract.html
- Qu M, Chen J, Xhonneux LP, et al (2021) RNNLogic: Learning logic rules for reasoning on knowledge graphs. In: International Conference on Learning Representations, URL <https://arxiv.org/abs/2010.04029>
- Schlichtkrull M, Kipf TN, Bloem P, et al (2018) Modeling relational data with graph convolutional networks. In: European Semantic Web Conference, pp 593–607, https://doi.org/10.1007/978-3-319-93417-4_38
- Shi B, Weninger T (2017) ProjE: Embedding projection for knowledge graph completion. In: Proceedings of the AAAI Conference on Artificial Intelligence, <https://doi.org/10.1609/aaai.v31i1.10677>
- Suchanek FM, Kasneci G, Weikum G (2007) YAGO: A core of semantic knowledge. In: Proceedings of the 16th International Conference on World Wide Web, pp 697–706, <https://doi.org/10.1145/1242572.1242667>
- Wang Q, Mao Z, Wang B, et al (2017) Knowledge graph embedding: A survey of approaches and applications. IEEE Transactions on Knowledge and Data Engineering 29(12):2724–2743. <https://doi.org/10.1109/tkde.2017.2754499>
- Wang X, Chen L, Ban T, et al (2021) Knowledge graph quality control: A survey. Fundamental Research 1(5):607–626. <https://doi.org/10.1016/j.fmre.2021.09.003>
- Wei J, Wang X, Schuurmans D, et al (2022) Chain-of-thought prompting elicits reasoning in large language models. In: Advances in Neural Information Processing Systems, pp 24824–24837, <https://doi.org/10.52202/068431-1800>
- White J, Fu Q, Hays S, et al (2023) A prompt pattern catalog to enhance prompt engineering with ChatGPT. In: Proceedings of the 30th Conference on Pattern Languages of Programs. ACM, <https://doi.org/10.64346/PLoP2023p05>
- Wienand D, Paulheim H (2014) Detecting incorrect numerical data in DBpedia. In: European Semantic Web Conference, pp 504–518, https://doi.org/10.1007/978-3-319-07443-6_34
- Yang B, Yih Wt, He X, et al (2015) Embedding entities and relations for learning and inference in knowledge bases. In: International Conference on Learning Representations, URL <https://arxiv.org/abs/1412.6575>
- Yao S, Zhao J, Yu D, et al (2023) ReAct: Synergizing reasoning and acting in language models. In: International Conference on Learning Representations, URL <https://arxiv.org/abs/2210.03629>

- Zaveri A, Rula A, Maurino A, et al (2016) Quality assessment for linked data: A survey. *Semantic Web* 7(1):63–93. <https://doi.org/10.3233/sw-150175>
- Zhu Z, Zhang Z, Xhonneux LP, et al (2021) Neural Bellman-Ford networks: A general graph neural network framework for link prediction. In: *Advances in Neural Information Processing Systems*, pp 29476–29490, URL https://papers.nips.cc/paper_files/paper/2021/hash/f6a673f09493afcd8b129a0bcf1cd5bc-Abstract.html